

Research Master R Programming Handout

Cas Goos

Felix Clouth

2026-07-09

Contents

1	About	5
2	Module 1: Setup	7
2.1	What is R?	7
2.2	Installing R	8
2.3	Installing RStudio	8
3	The Basics	9
3.1	Navigating RStudio	9
3.2	Data as Objects	12
3.3	Functions	17
4	Data	25
4.1	Types of Data	25
4.2	Reading & Writing Data	39
5	Cleaning & Summarizing Data	51
5.1	Recoding	51
5.2	Recoding with Nested Indexing	52
5.3	Functions in Functions	56
5.4	Doing Something to every row/column	58
6	Statistical Analysis	63
6.1	Formula Notation	63
6.2	Basic Statistical Tests	64

Chapter 1

About

This is a handout from another computer???

Chapter 2

Module 1: Setup

2.1 What is R?

R is a programming language and computational environment that is mainly used as ready-to-use software for statistical data analysis, both simple and advanced. Reasons for using R include:

1. It is free. R and all its packages (software that extends R's functionality to enable complex analyses and extended functionality) are absolutely free.
2. It's open. Science can only progress if there is complete openness about method. In SPSS and other commercial packages, it is not possible to find out exactly what the program is doing. Companies keep this a secret. All R's methods are on public display.
3. R is versatile. Statisticians who develop methods used by social scientists usually implement them in R as packages. Through packages, R's functionality extends far beyond many other software packages.
4. R is widely used. This means that colleagues in both Academia and Industry will be easier to collaborate with if you understand R. It also means that there are many resources available to consult to help you use R.
5. R is easy*. While R may initially seem difficult – and there is a steep initial learning curve – once you have completed these modules you will have passed most of this learning curve and learning more advanced topics in R will come relatively easily. Furthermore, compared to other programming languages like java and C++, R reads a lot more like English and will be easy to understand. Throughout these modules, you will make use of R in combination with Rstudio. While R is the programming language that does all the work of running the computations, Rstudio provides you with

an easy-to-use interface that makes working in R more comfortable and easier to understand.

2.2 Installing R

To install R for free, go to the following home website: <https://cran.r-project.org/>. Then:

- Under Download and Install R click on the hyperlink that corresponds to your operating system:
 - For Windows click on Download R for Windows, and then click on base and then click on the hyperlink at the top of the page (e.g., Download R-4.5.3 for Windows).
 - For Mac click on Download R for macOS, and then click on the top link (e.g., R-4.5.3-arm64.pkg).
- After downloading the R file, click to install it. Choose default answers for all the questions in the installer.

2.3 Installing RStudio

To install RStudio for free, go to the following home website: <https://posit.co/download/rstudio-desktop>. Then:

- If you are on Windows click on the Download Rstudio Desktop for Windows button.
- If you are on Mac, scroll down and click on the download link next to the macOS 14/15/16 (e.g. RStudio-2026.04.0-526.dmg).
- After downloading the RStudio file, click to install it. Choose default answers for all the questions in the installer.

Now that you have successfully installed R and RStudio, open RStudio (we will only access R through Rstudio for the remainder of these modules).

Chapter 3

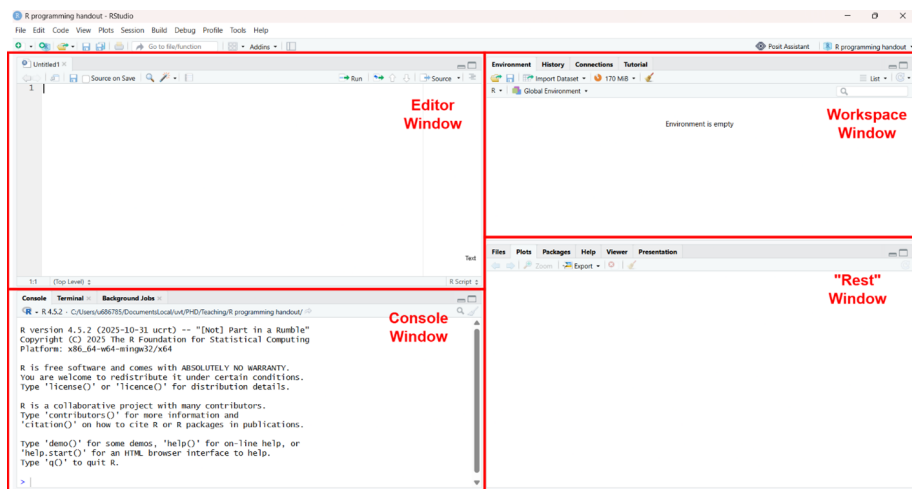
The Basics

Download link for Exercise 1: [exercise 1](#)

Download link for Answers to Exercise 1: [solutions exercise 1](#)

3.1 Navigating RStudio

RStudio is laid out into 4 different panes. The default panes are as shown below:

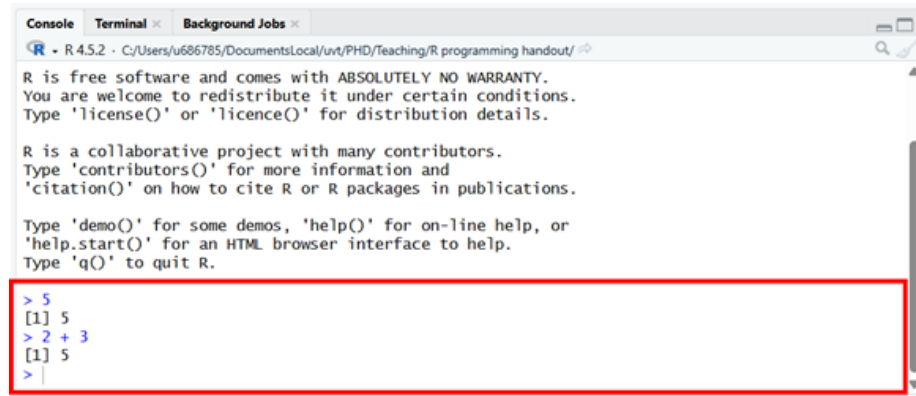


3.1.1 Console Window

- This window is where your code will be run by R, and also where the result of the code will be printed.

- You can try it out by typing 5 into the R console after the “>” prompt, and then hitting enter. Then type $2 + 3$. What do you see?

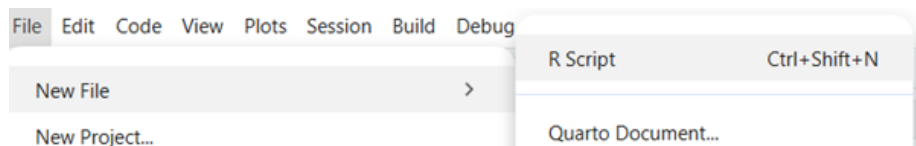
In both cases R returns the 5 as a response from the input.



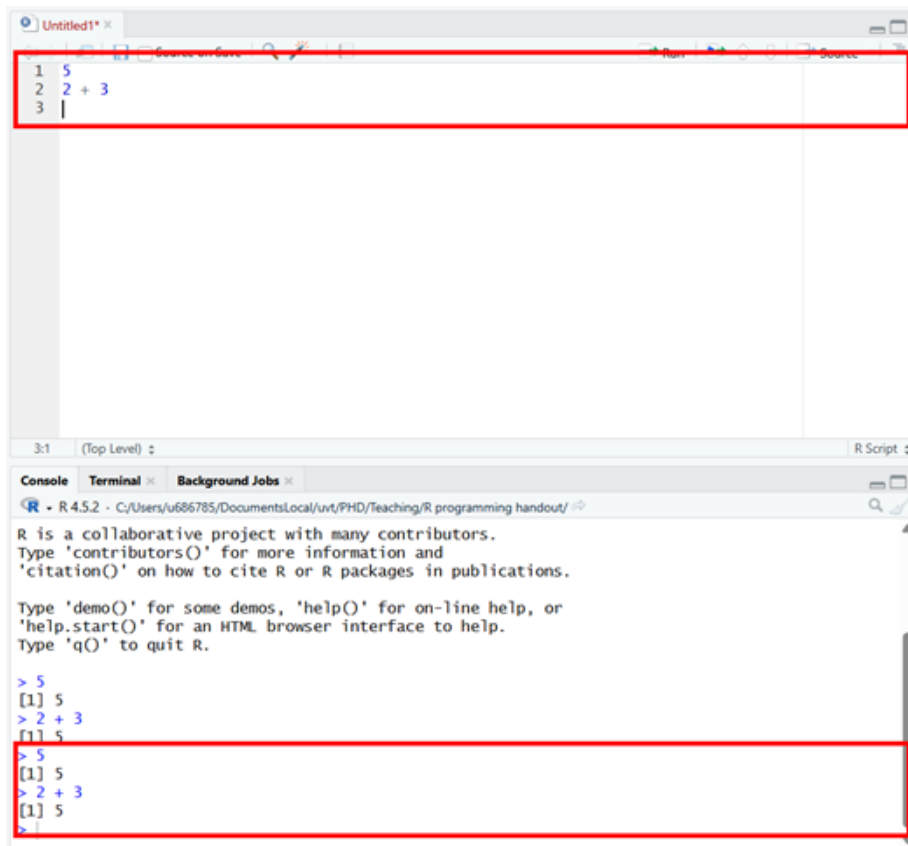
```
R is free software and comes with ABSOLUTELY NO WARRANTY.  
You are welcome to redistribute it under certain conditions.  
Type 'license()' or 'licence()' for distribution details.  
  
R is a collaborative project with many contributors.  
Type 'contributors()' for more information and  
'citation()' on how to cite R or R packages in publications.  
  
Type 'demo()' for some demos, 'help()' for on-line help, or  
'help.start()' for an HTML browser interface to help.  
Type 'q()' to quit R.  
  
> 5  
[1] 5  
> 2 + 3  
[1] 5  
> |
```

3.1.2 Editor Window

- This window is where you can create R scripts wherein you can write down your code to store so you can rerun it multiple times and store it for later reuse.
- To open an R script, navigate to the following:



- In your newly opened R script, type in 5 on line 1, then click enter to create a new line and type $2 + 3$. To run these lines of codes, select them or hold your cursor on that line and either press CTRL+ENTER, or click on the Run button. What do you see?



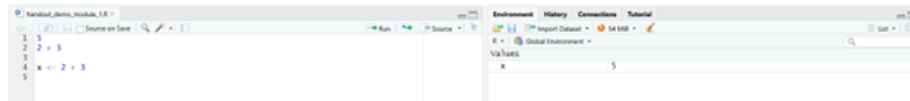
The commands are run through the console, and the output is presented the same way as before.

- To store this file either press CTRL+S, or navigate to File > Save, and give the script a name. You can reopen this file and your previously written code will be available for you to rerun.

3.1.3 Workspace Window

- This window contains your Environment which shows all the Objects (more on these later) you currently have stored in your R workspace. This window shows you what information you currently have stored.
- In your Editor Window type `x <- 2 + 3` and run it. What do you see in your Workspace window?

You can see that the Object `x` is now stored in your workspace and has the value of 5 assigned to it.



- Now type `x` into your Editor Window and run it, what output do you see in the console window?

The console returns 5, the value of `x`.

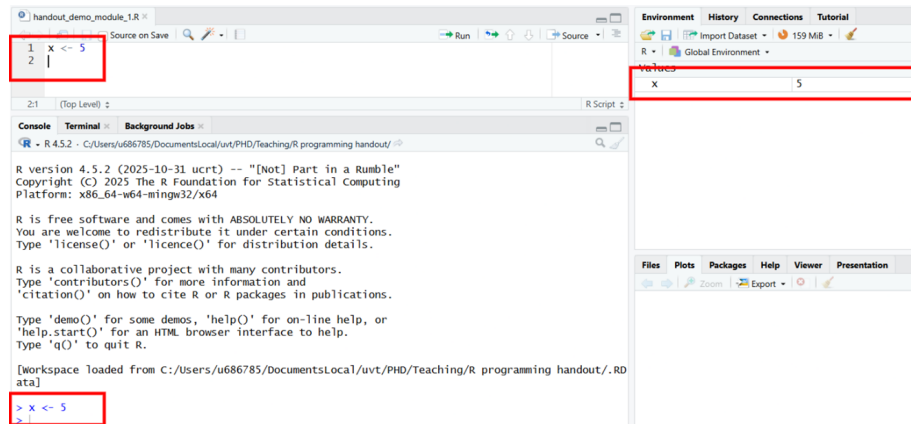
3.1.4 “Other” Window

This window contains several different panes each with their own function, we will encounter some of these panes as we go through these modules and some others you will likely never use for the rest of your life.

START EXERCISE 1.1

3.2 Data as Objects

In R, we spend most of our time working with objects. Objects can be almost anything, from numbers like the 5 you worked with earlier, to whole datasets, graphic figures, and even code itself can be stored and passed around as an object. To create your own object you make use of `<-` as you did previously:



As a quick note: we advise you to code along with these examples so you understand how to implement them in RStudio.

One major type of object you will frequently encounter (and in fact you just created one) is Data. Data is a broad category comprising anything from a single number to a multi-dimensional array, but for now let's focus on vectors.

Table 3.1: Fundamental R Data Types

Example	Type	Explanation
42	Integer (numeric)	A number contains no decimals
2.1543	Double (numeric)	A number that <i>may</i> contain decimals
"Felix"; "Tilburg"	Character	A string of textual characters
TRUE; FALSE	Logical	Special characters indicating either TRUE or FALSE

3.2.1 Vectors

A vector is *one or more of the same type of things, one after the other*

The function `c()` can create vectors:

```
c(1, 2, 3)
```

```
## [1] 1 2 3
```

```
c("Hagrid", "Harry")
```

```
## [1] "Hagrid" "Harry"
```

Notice I make vectors of numbers or sentences but not both at the same time - hence “the same type of thing”. Below are examples of different “single” objects that can occur together in a vector.

Note that you cannot create a vector from different types of objects. Thus in the following case:

```
c("Hagrid", 1)
```

```
## [1] "Hagrid" "1"
```

R automatically turns the 1 numeric type variable into a "1" character type variable to turn the vector into a character vector.

You may hear some people call R a “vectorized language”. What they are referring to is that when you perform an operation on a vector in R it will often perform that operation on each element of the vector.

For example, suppose there are 14 students in a class, of the following ages:

```
c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
```

```
## [1] 22 22 21 25 23 19 23 25 19 21 20 23
```

Then I can get their squared ages by:

```
c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)^2
```

```
## [1] 484 484 441 625 529 361 529 625 361 441 400 529
```

Or the sum of the ages as:

```
sum(c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23))
```

```
## [1] 263
```

Typing out a vector each time you want to do something with it is way too much work and looks messy. Just like any other object in R, you can store a vector of values for later use by assigning it to a variable. This works the same way as assigning a single value:

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
ages
```

```
## [1] 22 22 21 25 23 19 23 25 19 21 20 23
```

3.2.2 Variables

A variable is a placeholder name that refers to some object, such as a vector. They function like the numbers they have on clothing racks in some places:

- The coat is the value and the number on the hanger is the name of the variable
- So, `c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)` is the coat, and `ages` is the “hanger”.

Putting the coat on a hanger is called “assigning” a value to a variable. As mentioned before assigning value into variables is done with `<-`.

A variable can be named anything as long as it does not start with a number, does not contain operators, contains at least one regular letter character, and is preferably not already in use by R.

Table 3.2: Object Naming Rules

Acceptable	Not.Acceptable
my_var_3	3_my_var (starts with a number)
x_ (fine albeit ugly)	x^ (contains an operator character)
hamburgers_with_cheese (you can go as crazy as you want with your names)	_ (no letter character)
x	pi (already in use by R; meaning

```
pi
```

```
## [1] 3.141593
```

```
pi <- 5
pi
```

```
## [1] 5
```

To reset a workspace -for example in case of the assign to pi issue described above – you can make use of the following line of code:

```
rm(list = ls())
```

- With ls() you can see your objects
- With rm() you can remove them

Once you have assigned a value to your variable, you can now reuse that variable to perform all sorts of calculations.

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
ages^2
```

```
## [1] 484 484 441 625 529 361 529 625 361 441 400 529
```

```
sum(ages)
```

```
## [1] 263
```

```
220 - ages
```

```
## [1] 198 198 199 195 197 201 197 195 201 199 200 197
```

```
predicted_maximum_heart_rate <- 220 - ages
```

```
predicted_maximum_heart_rate
```

```
## [1] 198 198 199 195 197 201 197 195 201 199 200 197
```

Note: variable names are case sensitive!

ages is not the same as Ages or AGES or ageS!

If you misspell the variable name, R will respond with an error explaining what went wrong:

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
```

```
Ages
```

```
## Error:
## ! object 'Ages' not found
```

```
AGES
```

```
## Error:
## ! object 'AGES' not found
```

```
ageS
```

```
## Error:
## ! object 'ageS' not found
```

```
ages
```

```
## [1] 22 22 21 25 23 19 23 25 19 21 20 23
```

START EXERCISE 1.2

3.3 Functions

Besides Variables, another common object in R are Functions. Functions in R take the following form:

```
f()
```

You have already used several functions `c(1, 2, 3)`; `sum(ages)`; `rm()`; `ls()`.

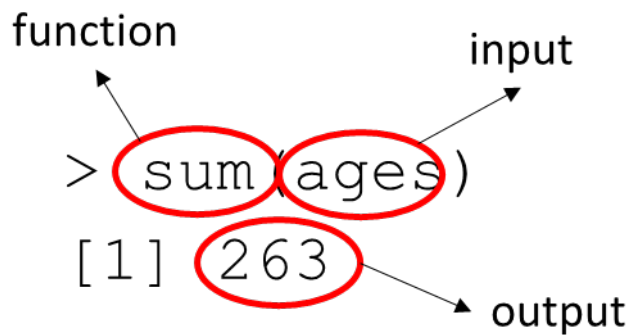
Another example of a function you have used already are operators such as `+` and `^`. You have used these operators as `3 + 2`. This is shorthand for `'+'(3, 2)`.

Almost always in R, functions also take some form of input, and have the following property:

You put something in, and you get something out

An example of this principle is the following:

```
## [1] 263
```



`sum` is a function that takes a vector (input) and returns the sum of that vector (output).

Another example is the following:

```
rnorm(10)
```

```
## [1] -1.30229509  0.01518197  0.85224807  0.29649831  1.29141551 -0.74137741
## [7]  0.61673299  1.18006506  0.14702770  0.82550108
```

```

> rnorm(10)
[1] 0.5681278 -0.2106897 2.8077816 1.6333306 -1.0294074 1.2491614 1.2058661
[8] 0.7381147 0.4392563 -0.7737585

```

`rnorm` is a function that takes an integer (input) and returns that number of random observations from a standard normal distribution (output).

The input of a function is called its arguments. Looking at our previous examples:

- `ages` is the argument in `sum(ages)`
- `10` is the argument in `rnorm(10)`

Functions can have multiple arguments:

```
rnorm(10, mean = 4, sd = 2)
```

```
## [1] 4.2836672 0.6754685 2.2787599 2.0931750 5.1978282 6.0300515 4.6291379
## [8] 3.5750516 5.9949502 5.0133451
```

Arguments can be specified either by name, or position:

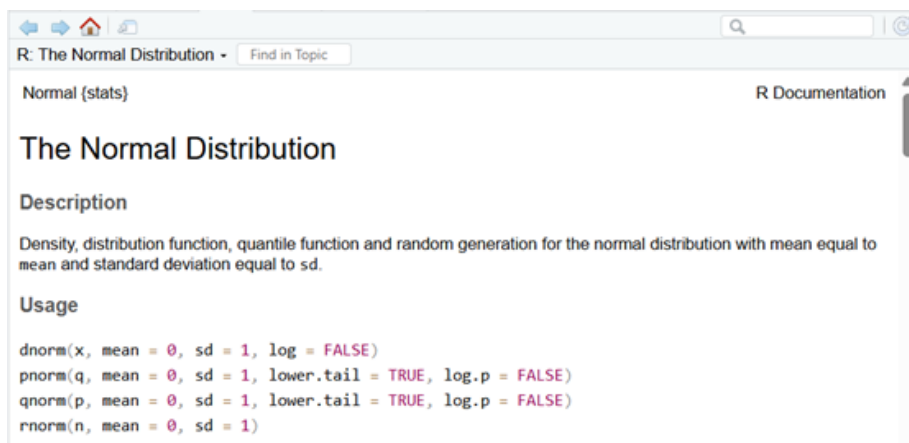
- `rnorm(10, mean = 4, sd = 2)` does the same as `rnorm(10, 4, 2)`.

While specifying by position is faster to write and can sometimes remain easy to read (note how we did not name the first argument in the `rnorm()` function above, for example), for clarity it is often smart to specify all arguments but the first by name.

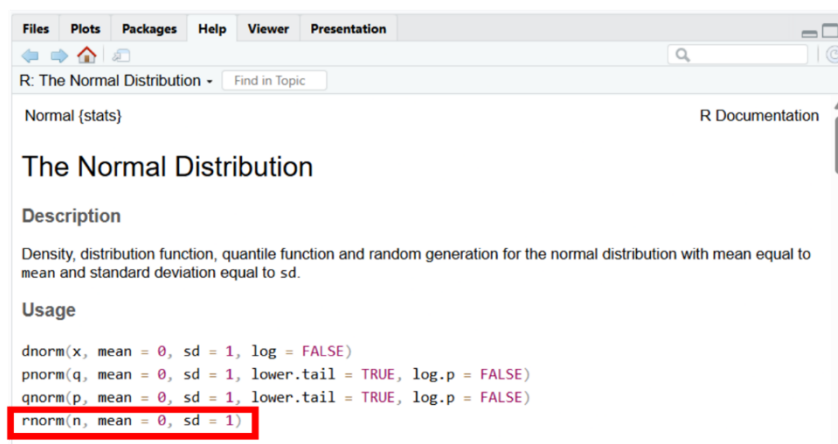
You can figure out which arguments you can specify and under which names by consulting the help files by adding `?` in front of the function name:

```
?rnorm
```

```
## starting httpd help server ... done
```



This opens the help files, which shows a description of the function, a description of its arguments, as well as default values for those arguments. Default values represent the values that the function will use for arguments when they are not specified.



We can see that for `rnorm`, the mean and standard deviation of the normal distribution from which `n` observations are drawn is 0 and 1 respectively. This means that if we don't specify `mean` and `sd`, and run `rnorm(10)`, we will draw 10 observations from a normal distribution with a mean of 0 and a standard deviation of 1.

We can also see that `n` does not have a default argument, which means that when we do not specify it, R gives us the following error:

```
rnorm(mean = 4, sd = 2)
```

```
## Error in `rnorm()`:

```

```
## ! argument "n" is missing, with no default
```

Some arguments have to be specified for a function to run. How do you know which arguments need to be specified? Look at the help files (or online).

Another common reason for errors is the result of inserting the wrong data type (integer, float, character, logical, etc.) into an argument. The `sum()` function in the following example requires a numeric vector as input, but we gave it the character vector `sum(c("Happy Meal", "Roger Rabbit"))` instead:

```
sum(c("Happy Meal", "Roger Rabbit"))
```

```
## Error in `sum()`:
## ! invalid 'type' (character) of argument
```

The error message shows that an invalid type of argument was used. Logically this makes sense too, you cannot calculate a numeric sum from "Happy Meal" & "Roger Rabbit". Again, look at the help files (or online) to learn what type of arguments are valid input.

Some more example functions:

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
length(ages)
```

```
## [1] 12
```

`length()` prints out the number of elements within a vector.

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
table(ages)
```

```
## ages
## 19 20 21 22 23 25
##  2  1  2  2  3  2
```

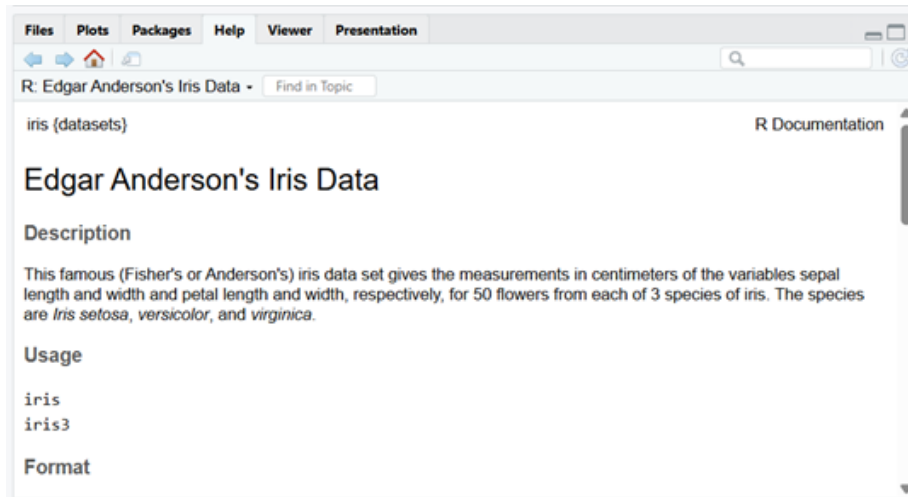
`table()` provides a descriptive overview of each unique value in a vector with a count showing how often the value occurred in the vector.

3.3.1 Finding Help

When using R, half your time will be spent searching help.

3.3.1.1 Help Files in R

We already showed you how you can use the `?` operator to open the help files for a function. It also works for built-in datasets in R:



You can also get help using the following functions:

- `help("sum")` or `help(sum)`
- `apropos("sum ")` or `apropos(sum) <-` finds all functions with “sum” in the name.
- `help.start()` to open the manual browser

3.3.1.2 Internet

You can make use of information available on the internet with the following resources:

- Google (or any other search engine)
- Stackoverflow (a website where a community of programmers help* answer each others question; *the tone isn't always the most helpful, but the information usually is)
- Cheat sheets, and other materials like these modules.
 - Basic: <https://cran.r-project.org/doc/contrib/Short-refcard.pdf>
 - For an overview, check <https://paulvanderlaken.com/2017/08/10/r-resources-cheatsheets-tutorials-books/>

3.3.1.3 Warnings & Errors

An error means that your code does not run, because it is wrongly specified. For example:

```
sum(c("Happy Meal", "Roger Rabbit"))
```

```
## Error in `sum()`:  
## ! invalid 'type' (character) of argument
```

A warning means that your code will run, but probably didn't result in the behaviour you expected. For example:

```
as.numeric(c(1, 2, "three", 4))
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 1 2 NA 4
```

In either case, do not ignore warning messages and errors. Even if everything is fine for now, you will likely run into issues later, unless you really know what you are doing.

To resolve an error/warning first carefully read it.

```
sum(c("Happy Meal", "Roger Rabbit"))
```

```
## Error in `sum()`:  
## ! invalid 'type' (character) of argument
```

To understand better where the origin of our error lies, we can separate our code into pieces:

```
vec_x <- c("Happy Meal", "Roger Rabbit")  
sum(vec_x)
```

```
## Error in `sum()`:  
## ! invalid 'type' (character) of argument
```

The fault is in the second line. The error message tells you what is wrong. You cannot use the `sum()` function on a character vector.

If the error or warning message is too complex to decipher yourself, a good second step is to type the error or warning message into Google, and then look for solutions others have proposed (Stackoverflow is again a particularly helpful resource here).

START EXERCISE 1.3

3.3.2 Indexing

Indexing in R refers to selecting a subset of elements from an object. Returning to our `ages` example:

```
ages <- c(22, 22, 21, 25, 23, 19, 23, 25, 19, 21, 20, 23)
```

What if we only wanted the 4th student's age?

In that case, you can use the following code:

```
ages[4]
```

```
## [1] 25
```

3.3.2.1 Indexing multiple elements

You index an element from a vector in R by placing the location of the element you want to index between square brackets [``...``] after the vector.

The [`...`] can also take vectors, in case you want more than one element from the vector:

```
ages[c(1, 3, 4)]
```

```
## [1] 22 21 25
```

In case you want the second through fifth elements of a vector:

```
ages[c(2, 3, 4, 5)]
```

```
## [1] 22 21 25 23
```

It can be easy to use the `:` operator.

```
ages[2:5]
```

```
## [1] 22 21 25 23
```

The `:` operator creates a vector running from the integer at the left side of the `:`, to the integer at the right side, in steps of 1:

```
2:5
```

```
## [1] 2 3 4 5
```

3.3.2.2 Negative Indexing

You can also index elements by telling R which element(s) you do not want. This is done by putting the `-` sign in front of a single value or a vector of values.

```
ages[-4]
```

```
## [1] 22 22 21 23 19 23 25 19 21 20 23
```

```
ages[-c(1, 3, 5, 7, 9, 11)]
```

```
## [1] 22 25 19 25 21 23
```

3.3.2.3 Named vectors & indexing by name

You can give names to elements in a vector:

```
llama_age <- c("Arthur" = 14, "Zaphod" = 9)
llama_age
```

```
## Arthur Zaphod
##      14      9
```

Now the vector `llama_age` has names. You can check this by doing:

```
names(llama_age)
```

```
## [1] "Arthur" "Zaphod"
```

This allows you to index elements within that vector by adding their name in between the square brackets `[...]`.

```
llama_age["Zaphod"]
```

```
## Zaphod
##      9
```

START EXERCISE 1.4

Chapter 4

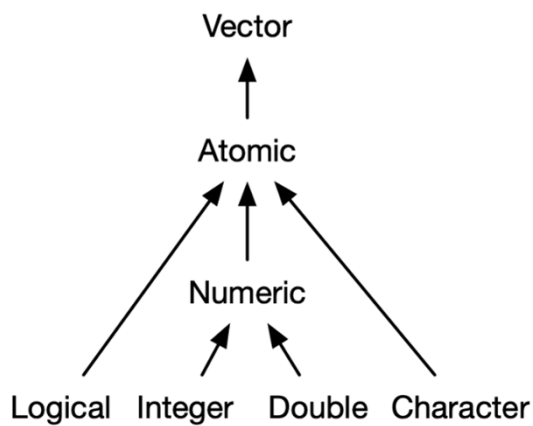
Data

Download link for Exercise 2 and the relevant Data: [exercise 2](#)

Download link for Answers to Exercise 2: [solutions exercise 2](#)

4.1 Types of Data

We have so far seen the following types of Data: Integer, Double, Character, and Logical. These can be organized as follows:



A special type of Integer is the Factor. A factor is a vector of values. The values refer to some category, the “levels”. For example:

```
gender <- factor(c("Male", "Female", "Female", "Female", "Male"),
               levels = c("Male", "Female"))
```

```
gender
```

```
## [1] Male   Female Female Female Male
## Levels: Male Female
```

Just like any other type of vector, a factor can be indexed in the same way:

```
gender[4]
```

```
## [1] Female
## Levels: Male Female
```

```
gender[1:3]
```

```
## [1] Male   Female Female
## Levels: Male Female
```

You can see the possible categories of the factor by using the `levels()` function:

```
levels(gender)
```

```
## [1] "Male"   "Female"
```

You can also *change* the categories of the factor values by using the `levels()` function:

```
levels(gender) <- c("Harry", "Sally")
```

```
gender
```

```
## [1] Harry Sally Sally Sally Harry
## Levels: Harry Sally
```

Under the hood, a factor is a vector of integers. Each integer number is then associated with a name (like Harry or Sally, or Male or Female), which is displayed as the output of the factor.

We can see the underlying numerical structure by using the `as.numeric()` function:

```
as.numeric(gender)
```

```
## [1] 1 2 2 2 1
```

4.1.1 Scalars, Vectors, & Matrices

4.1.1.1 Scalars

The most basic type of data is the scalar. Scalars hold one value at a time. We can use scalars as the building blocks for more complex data types. Effectively, a scalar is a vector of length 1. Examples of scalars:

```
knitr::include_graphics("images/DataGraph2.png")
```

```
> 4
```

```
[1] 4
```

```
> "Grumpy"
```

```
[1] "Grumpy"
```

```
> TRUE
```

```
[1] TRUE
```

4.1.1.2 Vectors

The next type of data is the vector. A vector holds multiple values at the same time, but consists of only one dimension. We have already encountered atomic vectors throughout the previous module. When we discuss vectors throughout these modules we are referring to atomic vectors, meaning all elements in the vector are of the same data type. Examples of vectors:

```
> c(1:9)
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
> c("Grumpy", "Sleepy", "Happy", "Dopey")
```

```
[1] "Grumpy" "Sleepy" "Happy" "Dopey"
```

```
> c(TRUE, TRUE, FALSE, TRUE, FALSE, FALSE)
```

```
[1] TRUE TRUE FALSE TRUE FALSE FALSE
```

4.1.1.3 Matrices

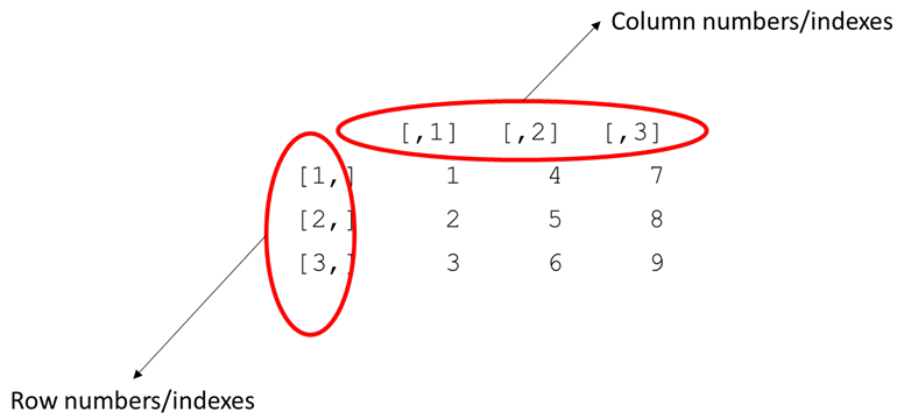
A matrix is an extension of the atomic vector. This means that every element in a matrix has to be of the same type. It functions similarly to a vector, only that each element in the matrix is organized along two dimensions. Example of a matrix:

```
my_matrix <- matrix(1:9, nrow = 3, ncol = 3)
```

```
my_matrix
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

Because a matrix is organised along two dimensions, you have to index it along both dimensions.



Indexing a matrix is done by indicating the row & column in the following way:

```
my_matrix[row_nr, col_nr]
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
my_matrix[1, ]
```

```
## [1] 1 4 7
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
my_matrix[, 1]
```

```
## [1] 1 2 3
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
my_matrix[, 1:2]
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
my_matrix[1, 3]
```

```
## [1] 7
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
my_matrix[3, 1]
```

```
## [1] 3
```

4.1.2 Lists & Data frames

4.1.2.1 Lists

So far, we have only looked at Atomic data. Atomic data requires all elements in it to be of the same type. The other category of data is List data. In Lists elements can be of different types. Here is an example:

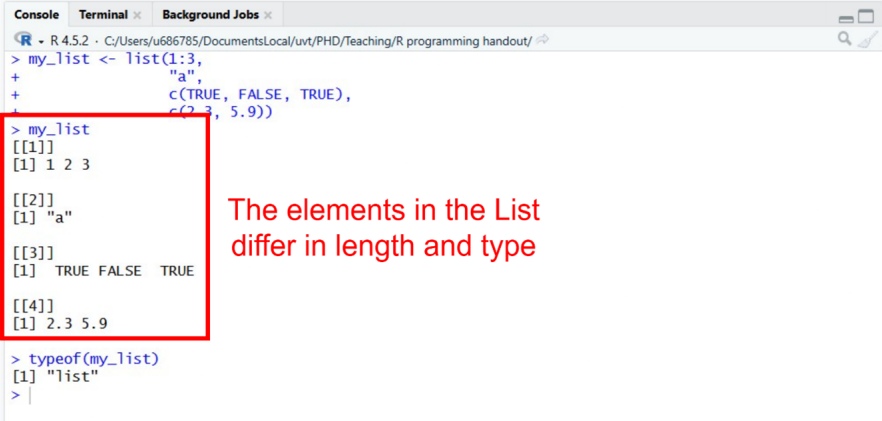
```
my_list <- list(1:3,
               "a",
               c(TRUE, FALSE, TRUE),
               c(2.3, 5.9))
```

```
my_list
```

```
## [[1]]
## [1] 1 2 3
##
```

```
## [[2]]
## [1] "a"
##
## [[3]]
## [1] TRUE FALSE TRUE
##
## [[4]]
## [1] 2.3 5.9
```

You add elements to a list by creating objects as you would any other object and separating them by a `,`. These objects can be of any type or length.



The screenshot shows an R console window with the following code and output:

```
> my_list <- list(1:3,
+               "a",
+               c(TRUE, FALSE, TRUE),
+               c(2.3, 5.9))
> my_list
[[1]]
[1] 1 2 3

[[2]]
[1] "a"

[[3]]
[1] TRUE FALSE TRUE

[[4]]
[1] 2.3 5.9

> typeof(my_list)
[1] "list"
>
```

A red rectangle highlights the output of `my_list`, showing four elements of different lengths and types. To the right of the console output, red text reads: "The elements in the List differ in length and type".

4.1.2.2 Data frames

Data frames are a special type of list consisting entirely of named vectors, where every element within the list must be of the same length. Additionally, because it is made up of vectors, every element within each of these vectors must be of the same type.

Data frames are similar to the datasets you are used to with rows and columns. A data frame is constructed using the `data.frame()` function. Data frames are created similarly to lists. Usually when we construct a data frame, we name the vectors we add as elements in the following way:

```
StarWars <- data.frame(name = c("Luke", "C-3P0", "R2-D2", "Darth Vader", "Leia"),
                      height = c(172, 167, 96, 202, 150))

StarWars

##           name height
```

```
## 1      Luke    172
## 2      C-3PO   167
## 3      R2-D2    96
## 4  Darth Vader  202
## 5      Leia    150
```

The vectors added as elements into the data frame form the columns of the data frame. The rows are the values for each column at a particular horizontal index

	name	height
1	Luke	172
2	C-3PO	167
3	R2-D2	96
4	Darth Vader	202
5	Leia	150

Row 1

	name	height
1	Luke	172
2	C-3PO	167
3	R2-D2	96
4	Darth Vader	202
5	Leia	150

Column 1

You can index a data frame in the same as a matrix:

```
my_data_frame[row_nr, col_nr]
```



```
StarWars[2, 1]
```

```
## [1] "C-3PO"
```

	¹ name	² height
1	Luke	172
2	C-3PO	167
3	R2-D2	96
4	Darth Vader	202
5	Leia	150

You can also index columns using the \$ sign, like: `my_data_frame$variable`

```
StarWars$height
```

```
## [1] 172 167 96 202 150
```

	name	height
1	Luke	172
2	C-3PO	167
3	R2-D2	96
4	Darth Vader	202
5	Leia	150

You can also use the \$ *add* columns.

Table 4.1: Indexing a Data Frame

Indexing	What
StarWars[5, 1]	We index the element in the 5th row from the 1st column.
StarWars[5, "name"]	We index the element in the 5th row from the "name" column.
StarWars\$name[5]	We index the "name" column, and from that we index the 5th element.

```
StarWars$evil <- c(FALSE, FALSE, FALSE, TRUE, FALSE)
```

```
StarWars
```

```
##      name height  evil
## 1    Luke    172 FALSE
## 2   C-3PO    167 FALSE
## 3   R2-D2     96 FALSE
## 4 Darth Vader   202  TRUE
## 5    Leia    150 FALSE
```

There are thus multiple ways to index the same element from a data frame. Each of the following returns the "Leia" from the above data frame.

	1	2	3
	name	height	evil
1	Luke	172	FALSE
2	C-3PO	167	FALSE
3	R2-D2	96	FALSE
4	Darth Vader	202	TRUE
5	Leia	150	FALSE

4.1.2.3 Combining Elements in a List

You can combine all types of objects in a list. Every element within an object must be of the same type, but besides that all sorts of objects can be put into a list: scalars, vectors, matrices, data frames, yes even lists can go into lists (and then you can even put lists in those lists). For example:

```
SWList <- list(series = "Star Wars",
               movie_order = c(4, 5, 6, 1, 2, 3, 7, 8, 9),
               leads = data.frame(name = c("Luke", "C-3PO", "R2-D2", "Darth Vader",
               "Leia"),
               height = c(172, 167, 96, 202, 150),
               evil = c(FALSE, FALSE, FALSE, TRUE, FALSE)))

SWList
```

```
## $series
## [1] "Star Wars"
##
## $movie_order
## [1] 4 5 6 1 2 3 7 8 9
##
## $leads
##      name height  evil
## 1    Luke    172 FALSE
## 2   C-3PO    167 FALSE
## 3   R2-D2     96 FALSE
## 4 Darth Vader 202  TRUE
## 5    Leia    150 FALSE
```

```
$series
[1] "Star Wars"
```

Scalar

```
$movie_order
[1] 4 5 6 1 2 3 7 8 9
```

Vector

```
$leads
      name height  evil
1    Luke    172 FALSE
2   C-3PO    167 FALSE
3   R2-D2     96 FALSE
4 Darth Vader 202  TRUE
5    Leia    150 FALSE
```

Data frame

In practice, lists are often used to structure a function's output. In other words, if you run a statistical analysis, you will get different types of output, each stored in a different item of a list.

If you do not name my objects, R will give them numbers in double square brackets.

```
list("Star Wars",
     c(4, 5, 6, 1, 2, 3, 7, 8, 9),
     data.frame(name = c("Luke", "C-3P0", "R2-D2", "Darth Vader", "Leia"),
                 height = c(172, 167, 96, 202, 150),
                 evil = c(FALSE, FALSE, FALSE, TRUE, FALSE)))
```

```
## [[1]]
## [1] "Star Wars"
##
## [[2]]
## [1] 4 5 6 1 2 3 7 8 9
##
## [[3]]
##      name height  evil
## 1      Luke    172 FALSE
## 2    C-3P0    167 FALSE
## 3    R2-D2     96 FALSE
## 4 Darth Vader   202  TRUE
## 5      Leia    150 FALSE
```

Just like with data frames you can use the `$` to index a list if the objects in it are named.

```
SWList$movie_order
```

```
## [1] 4 5 6 1 2 3 7 8 9
```

By using a `$` to index from a list, you are taking the object in the list you want to index out of the list and returning it as its own element unto the console. In the example above that means that we are returning to the console the vector `movie_order` and its contents.

If you index using square brackets `[...]`, you will actually return a list containing only the element you indexed. Thus:

```
SWList[2]
```

```
## $movie_order
## [1] 4 5 6 1 2 3 7 8 9
```

Returns a list containing only one object, the `movie_order` vector. We will demonstrate the importance of this difference when we look at nested indexing in a list.

Because lists can contain elements containing other elements (which in turn can contain more elements, etc.), it is at times necessary to index an element from within a list, and then to index again within the element you have just indexed. To make this more understandable, let us return to our example and attempt to index the movie order of the 3rd movie:

First, we index the `movie_order` variable from the list, as shown above:

```
SWList$movie_order
```

```
## [1] 4 5 6 1 2 3 7 8 9
```

```
# OR
```

```
SWList[[2]]
```

```
## [1] 4 5 6 1 2 3 7 8 9
```

Then we add another set of `[...]` after this first indexing to index within this vector:

```
SWList$movie_order[3]
```

```
## [1] 6
```

```
# OR
```

```
SWList[[2]][3]
```

```
## [1] 6
```

```
$series
```

```
[1] "Star Wars"
```

```
$movie_order
```

```
[1] 4 5 6 1 2 3 7 8 9
```

```
$leads
```

	name	height	evil
1	Luke	172	FALSE
2	C-3PO	167	FALSE
3	R2-D2	96	FALSE
4	Darth Vader	202	TRUE
5	Leia	150	FALSE

You can also do nested indexing of a data frame inside of a list in the following ways:

```
SWList$leads[2, 3]
```

```
## [1] FALSE
```

```
# OR
```

```
SWList[[3]][2, 3]
```

```
## [1] FALSE
```

```
$series
```

```
[1] "Star Wars"
```

```
$movie_order
```

```
[1] 4 5 6 1 2 3 7 8 9
```

```
$leads
```

	name	height	evil
1	Luke	172	FALSE
2	C-3PO	167	FALSE
3	R2-D2	96	FALSE
4	Darth Vader	202	TRUE
5	Leia	150	FALSE

However, note here that if we indexed our `movie_order` vector using only single square brackets like so:

```
SWList[2]
```

We could not index from the vector, as we have only created a new list containing one element and we are now indexing a third element in that list, which it does not have. Therefore, R returns an empty list:

```
SWList[2][3]
```

```
## $<NA>
```

```
## NULL
```

START EXERCISE 2.1

4.2 Reading & Writing Data

So far, we have created all the data that we have used ourselves from scratch in R. Usually, when we are analysing data in research, we make use of datasets from an external source. For example, a .csv file exported from Qualtrics containing all the responses of your participants. We will now look at how we can load in data from external sources into R, and then save them to another file.

4.2.0.1 Build-in datasets

First, there are datasets available in R. These can be accessed just like any other object. For example, the `sleep` dataset can be accessed by just typing `sleep`:

```
sleep
```

```
##      extra group ID
## 1      0.7      1  1
## 2     -1.6      1  2
## 3     -0.2      1  3
## 4     -1.2      1  4
## 5     -0.1      1  5
## 6      3.4      1  6
## 7      3.7      1  7
## 8      0.8      1  8
## 9      0.0      1  9
## 10     2.0      1 10
## 11     1.9      2  1
## 12     0.8      2  2
## 13     1.1      2  3
## 14     0.1      2  4
## 15    -0.1      2  5
## 16     4.4      2  6
## 17     5.5      2  7
## 18     1.6      2  8
## 19     4.6      2  9
## 20     3.4      2 10
```

We can index this dataset like any other data frame:

```
sleep$group
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
## Levels: 1 2
```

If you want to know more about the contents of the dataset, you can type `?sleep` just you would for a function.

```
?sleep
```

If you want to work with this dataset further, it is advisable to store it as a separate object. This avoids similar issues as demonstrated with `pi` in module 1.

```
sleep_data <- sleep
```

4.2.0.2 R Packages & Package Datasets

Sometimes R packages also come with their own built-in datasets. These are loaded in the same way as the built-in R datasets. To access these datasets you do need to have the respective package installed and activated.

R packages are extensions to R's functionality created by members of the R community. Most of the R packages you will work with are hosted on the Comprehensive R Archive Network ([CRAN](https://cran.r-project.org/)). Packages hosted on this archive can be installed using the following format of code:

```
install.packages("package_name")
```

 You will only need to install your package onto your system once.

However, each time you start an R session, you will need to activate the package using the following code:

```
library(package_name)
```

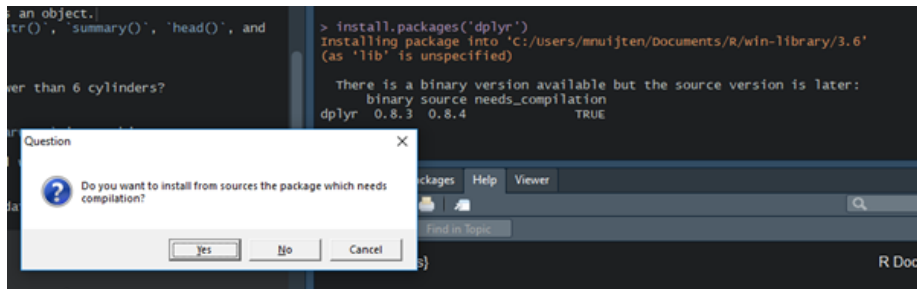
```
install.packages("dplyr") #only run this once
```

```
library(dplyr) #run this everytime you start an R session
```

It is generally advisable to run your `install.packages()` functions in the console and not store them in your script.

It is good coding standards to put ALL your library functions at the top of your R script.

Sometimes you will get the following pop-up when installing a package:



This means that there has been a recent update that is not available for your operating system yet. You have two options:

- “Yes”: build the latest version from source binary. Takes longer, you need extra packages, but you’ll have the latest version.
- “No”: build the previous version from binary files. A lot faster, no extra packages needed, but you won’t have the latest version (often not a problem).

4.2.0.3 Loading Data Files

You can also load in external files containing data into R. These will often be files exported from survey software like Qualtrics, or found through a data archive on the internet. Files that look like the following:

	A	B	C	D
1		extra	group	ID
2	1	0.7	1	1
3	2	-1.6	1	2
4	3	-0.2	1	3
5	4	-1.2	1	4
6	5	-0.1	1	5
7	6	3.4	1	6
8	7	3.7	1	7
9	8	0.8	1	8
10	9	0	1	9
11	10	2	1	10
12	11	1.9	2	1
13	12	0.8	2	2
14	13	1.1	2	3
15	14	0.1	2	4
16	15	-0.1	2	5
17	16	4.4	2	6
18	17	5.5	2	7
19	18	1.6	2	8
20	19	4.6	2	9
21	20	3.4	2	10

Table 4.2: Data Loading Functions

Function	Usage
<code>read.table()</code>	For text files
<code>read.csv()</code>	For comma separated spread sheets
<code>read.csv2()</code>	For comma separated spread sheets, but different decimal separators (see <code>?read.csv2</code>)
<code>load()</code>	For R data files
<code>read_spss()</code>	For SPSS files, from the package "foreign".
<code>read_excel()</code>	For Excel files, from the package "readxl"
<code>import()</code>	From the package "rio". Load any type of data (can be buggy)

You can load these types of files in using dedicated R functions. The function you need depend on the file type.

To load a file, you simply put the file path to the dataset relative to your working directory as a character string in the function name, as follows:

```
read.csv("data/sleep_data.csv")
```

To know where you have set your current working directory to, use the following code:

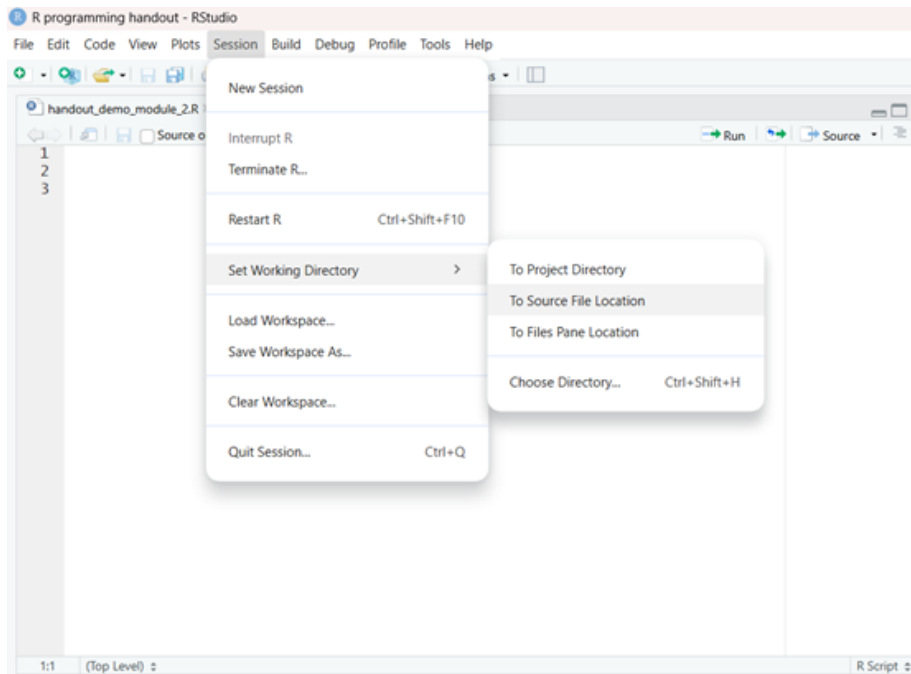
```
getwd()
```

To set your working directory to the location of your analysis script do:

```
knitr::include_graphics("images/SetwdGuide.png")
```

Table 4.3: Best Data Loading Functions

Function	Usage	File.Type
<code>read.table()</code>	For text files	.txt
<code>read.csv()</code>	For comma separated spread sheets	.csv
<code>read.csv2()</code>	For comma separated spread sheets, but different decimal separators (see <code>?read.csv2</code>)	.csv
<code>load()</code>	For R data files	.RData



Note: if the first row in the dataset contains the variable names, you will often have to specify the header argument as: `header = TRUE`.

The following functions are generally preferred, because they are stable, platform independent, and the result from these functions will be consistent.

To save a data set from R to a data file, you can use the following corresponding functions:

To save/write a file, you first put the name of the data file object you want to save. Then you put the file path to the directory where you want to store the dataset as a character string in the function, as follows:

```
write.csv(sleep_data, "data/sleep_data.csv")
```

Table 4.4: Best Data Loading Functions

Function	Usage
<code>write.table()</code>	For text files
<code>write.csv()</code>	For comma separated spread sheets
<code>write.csv2()</code>	For comma separated spread sheets, but different decimal separators (see <code>?write.csv2</code>)
<code>save()</code>	For R data files

4.2.1 Inspecting & Summarizing data

The first step after loading a dataset into your statistical software should always be to check your data, to see if it looks right.

- Are all variables there?
- Are all rows there?
- Are there any weird strings?
- Are there missing values (where I didn't expect them)?
- Do my variables have the correct names?
- ...

4.2.1.1 Summary Functions

There are several ready-made functions that will help you quickly investigate

- `dim()` # the dimensions of the data
- `nrow()` # the number of rows in the data frame / matrix
- `ncol()` # the number of columns in the data frame / matrix

```
dim(sleep)
```

```
## [1] 20 3
```

```
nrow(sleep)
```

```
## [1] 20
```

```
ncol(sleep)
```

```
## [1] 3
```

```
sleep
```

```
##      extra group ID
## 1      0.7      1  1
## 2     -1.6      1  2
## 3     -0.2      1  3
## 4     -1.2      1  4
## 5     -0.1      1  5
## 6      3.4      1  6
## 7      3.7      1  7
## 8      0.8      1  8
## 9      0.0      1  9
## 10     2.0      1 10
## 11     1.9      2  1
## 12     0.8      2  2
## 13     1.1      2  3
## 14     0.1      2  4
## 15    -0.1      2  5
## 16     4.4      2  6
## 17     5.5      2  7
## 18     1.6      2  8
## 19     4.6      2  9
## 20     3.4      2 10
```

To evaluate vectors, the following functions are useful:

We have already seen:

- `mean()`
- `median()`
- `sum()`
- `length()`

To find the highest or lowest value in a vector, the following functions can be used:

- `max()`

- `min()`

```
max(c(3, 1, 4, 3))
```

```
## [1] 4
```

To find the location of the highest or lowest value in a vector, the following functions can be used:

- `which.max()`
- `which.min()`

```
which.max(c(3, 1, 4, 3))
```

```
## [1] 3
```

The use of other summarizing functions like `View()`, `summary()`, `str()`, `head()`, will be explored in Exercise 2.2.

4.2.1.2 Conditional Selection

So far, when we have indexed datasets or vectors, we have made use of names (e.g., `llama_age['Zaphod']`, or `StarWars$height`), or using numeric values to indicate locations (e.g., `ages[c(1, 3, 4)]`). We can also use vectors of logical (TRUE/FALSE) values for indexing.

To index with a vector of logical values the vector must be the same length as the object you want to index over.

Let's say for example, we wanted to use logical values to index over the `sleep` dataset, which has 20 rows and 3 columns.

```
dim(sleep)
```

```
## [1] 20 3
```

To index the second column of this dataset using logical indexing, we would use:

```
c(FALSE, TRUE, FALSE)
```

```
sleep[, c(FALSE, TRUE, FALSE)]
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2
## Levels: 1 2
```

To index over rows would require a vector of length 20. This would take a lot of work to type out. Fortunately, we can construct logical vectors using conditional statements as in the following example:

```
sleep$extra > 0
```

This returns TRUE if at that position the value of extra is larger than 0, and FALSE otherwise.

```
sleep$extra
```

```
## [1] 0.7 -1.6 -0.2 -1.2 -0.1 3.4 3.7 0.8 0.0 2.0 1.9 0.8 1.1 0.1 -0.1
## [16] 4.4 5.5 1.6 4.6 3.4
```

```
sleep$extra > 0
```

```
## [1] TRUE FALSE FALSE FALSE FALSE TRUE TRUE TRUE FALSE TRUE TRUE TRUE
## [13] TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE
```

To then index with this logical vector, we do the following:

```
sleep[sleep$extra > 0, ]
```

This means: return all rows in sleep, where the value of the variable extra in the sleep dataset is larger than 0.

```
sleep[sleep$extra > 0, ]
```

```
##      extra group ID
## 1    0.7      1  1
## 6    3.4      1  6
## 7    3.7      1  7
## 8    0.8      1  8
## 10   2.0      1 10
## 11   1.9      2  1
## 12   0.8      2  2
## 13   1.1      2  3
## 14   0.1      2  4
## 16   4.4      2  6
## 17   5.5      2  7
## 18   1.6      2  8
## 19   4.6      2  9
## 20   3.4      2 10
```

You can construct negative logical expressions by adding the `!` operator in front of the statement. This will turn all “TRUEs” into “FALSEs” and vice versa:

```
sleep$extra > 0
```

```
## [1] TRUE FALSE FALSE FALSE FALSE TRUE TRUE TRUE FALSE TRUE TRUE TRUE
## [13] TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE
```

```
!sleep$extra > 0
```

```
## [1] FALSE TRUE TRUE TRUE TRUE FALSE FALSE FALSE TRUE FALSE FALSE FALSE
## [13] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
```

You can combine logical expressions using the OR (`|`), and AND (`&`) operators. For example:

```
sleep$extra > 0 & sleep$group == 1
```

will return true for all cases for which the value of `extra` is larger than 0 and were in `group 1`.

```
sleep$extra > 0 & sleep$group == 1
```

```
## [1] TRUE FALSE FALSE FALSE FALSE TRUE TRUE TRUE FALSE TRUE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
sleep[sleep$extra > 0 & sleep$group == 1, ]
```

```
##      extra group ID
## 1    0.7      1  1
## 6    3.4      1  6
## 7    3.7      1  7
## 8    0.8      1  8
## 10   2.0      1 10
```

Testing for missing values works differently. In R missing values are commonly indicated by the special character `NA`. Because this is a special character certain standard operations will not work as expected. For instance:

```
sleep$extra == NA
```

returns


```
sleep$extra == NA
```

```
## [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
```

To test for missing values, we instead make use of the `is.na()` function.

```
is.na(sleep$extra)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Or for data with NAs:

```
airquality$Ozone
```

```
## [1] 41 36 12 18 NA 28 23 19 8 NA 7 16 11 14 18 14 34 6
## [19] 30 11 1 11 4 32 NA NA NA 23 45 115 37 NA NA NA NA
## [37] NA 29 NA 71 39 NA NA 23 NA NA 21 37 20 12 13 NA NA NA
## [55] NA NA NA NA NA NA NA 135 49 32 NA 64 40 77 97 97 85 NA
## [73] 10 27 NA 7 48 35 61 79 63 16 NA NA 80 108 20 52 82 50
## [91] 64 59 39 9 16 78 35 66 122 89 110 NA NA 44 28 65 NA 22
## [109] 59 23 31 44 21 9 NA 45 168 73 NA 76 118 84 85 96 78 73
## [127] 91 47 32 20 23 21 24 44 21 28 9 13 46 18 13 24 16 13
## [145] 23 36 7 14 30 NA 14 18 20
```

```
is.na(airquality$Ozone)
```

```
## [1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] TRUE TRUE TRUE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
## [37] TRUE FALSE TRUE FALSE FALSE TRUE TRUE FALSE TRUE TRUE FALSE FALSE
## [49] FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [73] FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE
## [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [97] FALSE FALSE FALSE FALSE FALSE TRUE TRUE FALSE FALSE FALSE TRUE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE
```

```
airquality$Ozone[!is.na(airquality$Ozone)]
```

```
##      [1]  41  36  12  18  28  23  19   8   7  16  11  14  18  14  34   6  30  11
##     [19]   1  11   4  32  23  45 115  37  29  71  39  23  21  37  20  12  13 135
##     [37]  49  32  64  40  77  97  97  85  10  27   7  48  35  61  79  63  16  80
##     [55] 108  20  52  82  50  64  59  39   9  16  78  35  66 122  89 110  44  28
##     [73]  65  22  59  23  31  44  21   9  45 168  73  76 118  84  85  96  78  73
##     [91]  91  47  32  20  23  21  24  44  21  28   9  13  46  18  13  24  16  13
##    [109]  23  36   7  14  30  14  18  20
```

Finally, here is a list of the common types of conditional selection operators you will be using:

- `>` (“greater than”)
- `<` (“smaller than”)
- `>=` (“greater than or equal to”)
- `<=` (“smaller than or equal to”)
- `==` (“equal to”)
- `!=` (“not equal to”)
- `is.na()` (“is missing”)
- `&` (“AND”)
- `|` (“OR”)

START EXERCISE 2.2

Chapter 5

Cleaning & Summarizing Data

Download link for Exercise 3 and the relevant Data: [exercise 3](#)

Download link for Answers to Exercise 3: [solutions exercise 3](#)

5.1 Recoding

In Module 2 we said that you should always check your data as the first step in your analyses. After checking your data, your second step will often be to clean your data. Raw data is often messy and requires you to treat missing values and errors in the data.

In some statistical analysis programs, you as a user specify what is a missing value is. These are often improbable values like 99.

a	b	c	d	e	f
7	5	99	2	5	2
5	5	5	3	6	1
6	8	5	9	9	4
4	2	2	6	6	8
6	7	6	99	10	6
9	99	4	7	5	1

R will treat these as numbers (because they are), not as missings. We need to recode these values to NAs.

```
vector <- c(4, 1, 6, 99, 9)
vector
```

```
## [1] 4 1 6 99 9
```

Supposing that the 99 should be a missing value, we recode that specific value into an NA by making use of indexing:

```
vector <- c(4, 1, 6, 99, 9)
vector
```

```
## [1] 4 1 6 99 9
```

```
vector[vector == 99] <- NA
vector
```

```
## [1] 4 1 6 NA 9
```

5.2 Recoding with Nested Indexing

For data cleaning it is often necessary to make use of complex nested indexing patterns to select the specific elements you want to change.

Let us demonstrate using the following sample data frame:

```
df <- data.frame(var1 = c(4, 1, 6, 99, 9),
                  var2 = c(99, 9, 1, 4, 2))
df
```

```
##   var1 var2
## 1    4   99
## 2    1    9
## 3    6    1
## 4   99    4
## 5    9    2
```

```
df == 99
```

```
##      var1 var2
## [1,] FALSE TRUE
## [2,] FALSE FALSE
## [3,] FALSE FALSE
## [4,]  TRUE FALSE
## [5,] FALSE FALSE
```

```
df[df == 99] <- NA
```

```
df
```

```
##   var1 var2
## 1    4   NA
## 2    1    9
## 3    6    1
## 4   NA    4
## 5    9    2
```

If we only wanted to change the 99 in the second column, we would do the following:

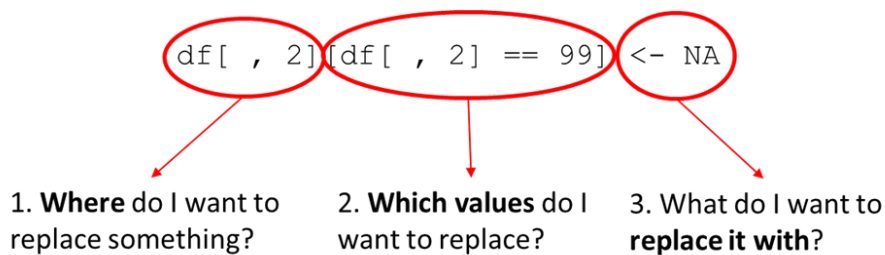
```
df <- data.frame(var1 = c(4, 1, 6, 99, 9),
                  var2 = c(99, 9, 1, 4, 2))
```

```
df[, 2][df[, 2] == 99] <- NA
```

```
df
```

```
##   var1 var2
## 1    4   NA
## 2    1    9
## 3    6    1
## 4   99    4
## 5    9    2
```

The structure of this nested indexing and data cleaning is as follows:



```
df[ , 2][df[ , 2] == 99] <- NA
```



```
x <- NA
```

“I want to assign the value NA to the object x”

```
df[ , 2][df[ , 2] == 99] <- NA
```



```
df[ , 2]
```

“we want to replace something in the second column of df”

```
# NA 9 1 4 2
```

```
df[ , 2][df[ , 2] == 99] <- NA
```



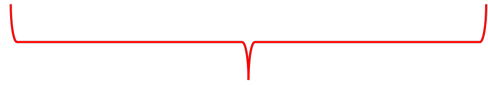
```
df[ , 2] == 99
```

“which elements of the second column of df have the value 99?”

```
# TRUE FALSE FALSE FALSE FALSE
```

“the first element has the value 99”

```
df[ , 2][df[ , 2] == 99] <- NA
```



```
df[ , 2][df[ , 2] == 99]
```

“replace something in the second column of df, but only select the elements that have the value 99 in the second column”

```
# 99
```

```
df[ , 2][df[ , 2] == 99] <- NA
```

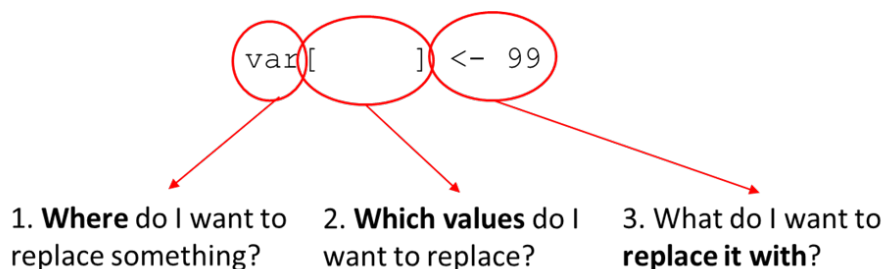


“replace something in the second column of df, but only select the elements that have the value 99 in the second column, and replace these with NA”

df

```
#   var1 var2
# 1     4  NA
# 2     1   9
# 3     6   1
# 4    99   4
# 5     9   2
```

If we wanted to do it the other way around, and recode NAs to other values such as 99, the structure of the indexing would remain the same. Thus, we would still follow the following format:



Only in this instance we have to make use of the `is.na()` function to detect NAs, because NAs are special characters that work differently:

```
var <- c(4, 1, 6, NA, 9)
```

```
var
```

```
## [1] 4 1 6 NA 9
```

```
var[var == NA] <- 99
```

```
var
```

```
## [1] 4 1 6 NA 9
```

As shown in the code above, nothing happens when we use `var == NA`.

```
var == NA
```

```
## [1] NA NA NA NA NA
```

We instead need to use the `is.na()` function

```
var
```

```
## [1] 4 1 6 NA 9
```

```
var[is.na(var)] <- 99
```

```
var
```

```
## [1] 4 1 6 99 9
```

In short:

 `var[var == NA] <- 99`

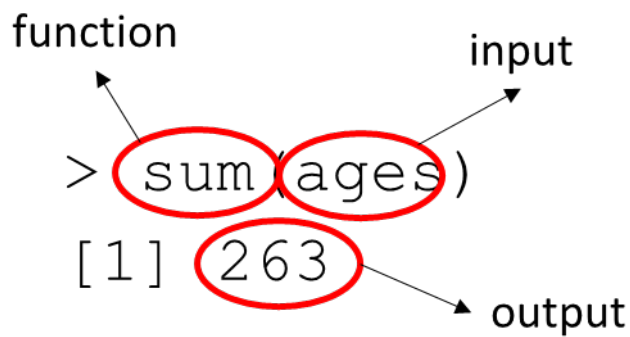
 `var[is.na(var)] <- 99`

START EXERCISE 3.1

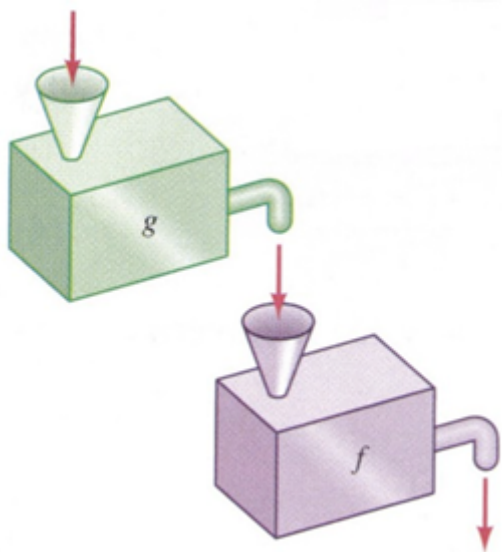
5.3 Functions in Functions

Remember, all functions take the following form:

You put something in, and you get something out



We can extend this principle to a sequence of functions where the output of one function becomes the input of another function. As illustrated here with the output of function `g` becoming the input of function `f`.



We can use this principle to change the following code:

```
df <- mtcars
dfhp <- df$hp
meanhp <- mean(dfhp)
round(meanhp, digits = 1)
```

```
## [1] 146.7
```

To this:

```
round(mean(df$hp), digits = 1)
```

```
## [1] 146.7
```

5.4 Doing Something to every row/column

One of the advantages of R we have already discussed is that it is a vectorized programming language. Meaning, that in most cases if an operation is performed on a vector it is performed on each element in that vector automatically, without the need for us to code our operation to work specifically on a vector.

Things get a little more complicated if we want to perform actions across every row and column in a data frame or matrix. This is not done automatically in R as for vectors. Luckily there are certain functions we can use that do allow us to perform operations across either rows or columns.

For example, when we want the mean of a vector we use the following operation:

```
vec <- c(6, 7, 3, 10)
```

```
mean(vec)
```

```
## [1] 6.5
```

If we want the mean of a data frame:

```
grades <- data.frame(course1 = c(6, 4, 8),  
                     course2 = c(7, 9, 8),  
                     course3 = c(3, 3, 6),  
                     course4 = c(10, 10, 9))
```

We can make use of the `colMeans()` function:

```
colSums(grades)
```

```
## course1 course2 course3 course4  
##      18      24      12      29
```

We can use similar functions for rows, but first let us name our rows to make the resulting output easier to read.

```
grades
```

```
##   course1 course2 course3 course4
## 1      6      7      3      10
## 2      4      9      3      10
## 3      8      8      6      9
```

```
row.names(grades) <- paste0("student", 1:3)
```

```
grades
```

```
##           course1 course2 course3 course4
## student1      6      7      3      10
## student2      4      9      3      10
## student3      8      8      6      9
```

The functions we use to get the means and sums for rows are `rowMeans()` and `rowSums()`:

```
rowMeans(grades)
```

```
## student1 student2 student3
##      6.50      6.50      7.75
```

```
rowSums(grades)
```

```
## student1 student2 student3
##      26      26      31
```

Taking means or sums is very common, but you could also be interested in obtaining something else, like the standard deviation for instance.

In that case you can use `apply()`, which is a function that can be used to “apply” any function over either rows or columns.

```
apply(grades, 2, mean)
```

```
##   course1 course2 course3 course4
## 6.000000 8.000000 4.000000 9.666667
```

```
colMeans(grades)
```

```
## course1 course2 course3 course4  
## 6.000000 8.000000 4.000000 9.666667
```

But we can also use other function besides `mean()`.

```
apply(grades, 2, median)
```

```
## course1 course2 course3 course4  
##      6      8      3      10
```

```
apply(grades, 2, sd)
```

```
## course1 course2 course3 course4  
## 2.000000 1.000000 1.7320508 0.5773503
```

The structure of the `apply()` function is as follows:

`apply(grades, 2, sd)`

`apply(grades, 2, sd)`

First, we name the object we want to apply a function over. In this case the `grades` data frame.

`apply(grades, 2, sd)`

Second, we specify the margin, meaning wherever we want to apply the function. In this case we specified the second margin which are the columns by choosing 2 as the input. Since there are 1) rows and 2) columns, the second margin is the columns.

The rows are the first margin, so:

```
apply(grades, 1, mean)
```

```
## student1 student2 student3
##      6.50      6.50      7.75
```

Is the same as:

```
rowMeans(grades)
```

```
## student1 student2 student3
##      6.50      6.50      7.75
```

Third, you specify the function you want to apply. In this case `sd()`, to calculate the standard deviations across all columns in `grades`. Note that we do not add the brackets `()` when we add the function as an argument in `apply` (otherwise it would try to enter the output of the function as the argument rather than the function itself as the argument).

```
apply(grades, 2, sd)
```

5.4.0.1 Applying over Groups

Suppose we had the following data frame with a factor variable,

```
grades2 <- data.frame(course1 = c(6, 4, 8, 6),
                      course2 = c(7, 9, 8, 7),
                      course3 = c(3, 3, 6, 7),
                      course4 = c(10, 10, 9, 2),
                      gender = factor(c("male", "male",
                                       "female", "female")))

row.names(grades2) <- paste0("student", 1:4)

grades2
```

```
##      course1 course2 course3 course4 gender
## student1      6      7      3      10  male
## student2      4      9      3      10  male
## student3      8      8      6       9 female
## student4      6      7      7       2 female
```

and we wanted to know the mean score and standard deviation for `course2` for each `gender` in our data frame.

In that case, we can make use of the `tapply()` function to apply a function for each group separately and return the output for each group.

```
tapply(grades2$course2, grades2$gender, mean)
```

```
## female    male  
##      7.5     8.0
```

```
tapply(grades2$course2, grades2$gender, sd)
```

```
##      female      male  
## 0.7071068 1.4142136
```

```
tapply(grades2$course2, grades2$gender, sd)
```

Similar to `apply()`, we first start with naming the object we want to apply our functions over. This should typically be a vector, so not a data frame unlike in `apply()`.

```
tapply(grades2$course2, grades2$gender, sd)
```

Second, we name the factor variable vector containing the groups across which we want to apply.

```
tapply(grades2$course2, grades2$gender, sd)
```

Third, we just like for `apply` name the function.

START EXERCISE 3.2

Chapter 6

Statistical Analysis

In this final module, we will look at how we can run some basic statistical functions in R. Fortunately, once the data has been properly loaded, checked, cleaned, and selected, running the statistical analysis is often as simple as running a single function with a handful of arguments.

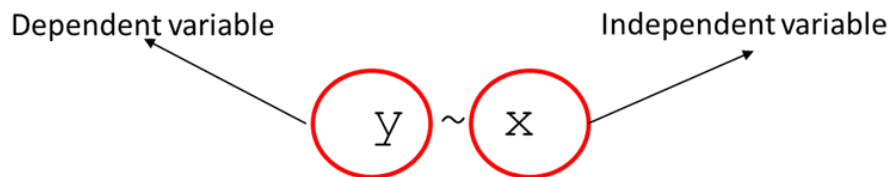
We will not be discussing the logic behind statistical analyses, or the meaning of these results. That will be the focus of the Multivariate Analysis course.

Download link for Exercise 4 and the relevant Data: [exercise 4](#)

Download link for Answers to Exercise 4: [solutions exercise 4](#)

6.1 Formula Notation

Most statistical functions in R require you to specify your model with formula notation, in the format: $y \sim x$.



The analysis function will then take this formula notation and run your model to analyze the relationship between these variables and return the resulting output in a list from which you can then index the relevant results to present (described in more detail later).

If you wanted to add multiple variables, you would do so in the following format:

Table 6.1: Formula Notation in R

Formula.in.R	Meaning
$y \sim 0 + x$ / $y \sim x - 1$	No intercept
$y \sim x + z$	Main effects
$y \sim x:z$	Interaction
$y \sim x*z$	Main & interaction effects /n Equivalent to: $y \sim x + z + x:z$
$y \sim \text{poly}(x, 2)$	Quadratic (2nd order polynomial)
$y \sim A$	ANOVA, A is a factor variable
$y \sim A + x$	ANOVA + covariate
$y \sim A*B*C - A:B:C$	No 3rd order interaction

$y \sim x1 + x2 + x3, \text{ etc.}$

If you wanted to add an interaction you would do so as follows:

$y \sim x1:x2$

Or equivalently:

$y \sim x1 + x2 + x1*x2$

As an overview, here is a table containing various common ways of specifying different formula notations.

6.2 Basic Statistical Tests

6.2.1 Linear Regression

The function used to conduct a linear regression in R is the `lm()` function. Typing `?lm` will show us all the arguments we can specify.

lm (stats)

R Documentation

Fitting Linear Models

Description

`lm` is used to fit linear models, including multivariate ones. It can be used to carry out regression, single stratum analysis of variance and analysis of covariance (although `gcv` may provide a more convenient interface for these).

Usage

```
lm(formula, data, subset, weights, na.action,
   method = "qr", model = TRUE, x = FALSE, y = FALSE, qr = TRUE,
   singular.ok = TRUE, contrasts = NULL, offset, ...)
```

```
## S3 method for class 'lm'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

formula an object of class `"formula"` (or one that can be coerced to that class): a symbolic description of the model to be fitted. The details of model specification are given under 'Details'.

data an optional data frame, list or environment (or object coercible by `as.data.frame` to a data frame) containing the

There are many potential arguments to specify. Luckily, we can leave most of these functions with the default. We should make sure to specify the following arguments: `formula` & `data`.

As an example, we will use the `iris` dataset, and try to regress petal width on petal length. We can do this as follows:

```
lm(Petal.Length ~ Petal.Width, data = iris)
```

or

```
lm(iris$Petal.Length ~ iris$Petal.Width)
```

```
lm(Petal.Length ~ Petal.Width, data = iris)
```

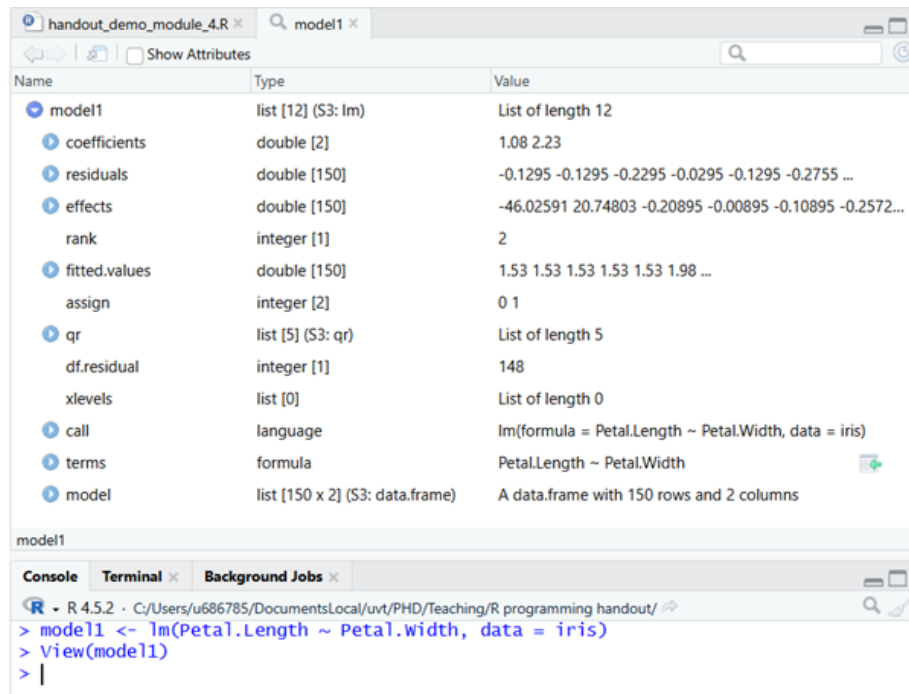
```
##
## Call:
## lm(formula = Petal.Length ~ Petal.Width, data = iris)
##
## Coefficients:
## (Intercept)  Petal.Width
##          1.084          2.230
```

We can see that we successfully ran the regression and we got our estimated coefficients (intercept and association between `Petal.Width` and `Petal.Length`).

However, we did not get any other results such as standard errors, t statistics, p-values, even degrees of freedom are missing.

If we want these additional results, we should first store our results as an object:

```
model1 <- lm(Petal.Length ~ Petal.Width, data = iris)
```



Looking at the `model1` object, we can see that it is a list containing the results as well as fitting information for the linear regression model we just ran.

Because these results are a list, we can access additional results from this output by indexing our `model1` object as we have done before:

```
model1$df.residual
```

```
## [1] 148
```

```
model1$residuals
```

```
##          1          2          3          4          5          6
## -0.129546132 -0.129546132 -0.229546132 -0.029546132 -0.129546132 -0.275534231
##          7          8          9         10         11         12
## -0.352540181 -0.029546132 -0.129546132  0.193447918 -0.029546132  0.070453868
##          13         14         15         16         17         18
##  0.093447918 -0.206552082 -0.329546132 -0.475534231 -0.675534231 -0.352540181
##          19         20         21         22         23         24
## -0.052540181 -0.252540181  0.170453868 -0.475534231 -0.529546132 -0.498528280
```

##	25	26	27	28	29	30
##	0.370453868	0.070453868	-0.375534231	-0.029546132	-0.129546132	0.070453868
##	31	32	33	34	35	36
##	0.070453868	-0.475534231	0.193447918	-0.129546132	-0.029546132	-0.329546132
##	37	38	39	40	41	42
##	-0.229546132	0.093447918	-0.229546132	-0.029546132	-0.452540181	-0.452540181
##	43	44	45	46	47	48
##	-0.229546132	-0.821522330	-0.075534231	-0.352540181	0.070453868	-0.129546132
##	49	50	51	52	53	54
##	-0.029546132	-0.129546132	0.494525274	0.071531224	0.471531224	0.017519323
##	55	56	57	58	59	60
##	0.171531224	0.517519323	0.048537175	-0.013498528	0.617519323	-0.305474726
##	61	62	63	64	65	66
##	0.186501472	-0.228468776	0.686501472	0.494525274	-0.382480677	0.194525274
##	67	68	69	70	71	72
##	0.071531224	0.786501472	0.071531224	0.363507423	-0.297450924	0.017519323
##	73	74	75	76	77	78
##	0.471531224	0.940513373	0.317519323	0.194525274	0.594525274	0.125543125
##	79	80	81	82	83	84
##	0.071531224	0.186501472	0.263507423	0.386501472	0.140513373	0.448537175
##	85	86	87	88	89	90
##	0.071531224	-0.151462825	0.271531224	0.417519323	0.117519323	0.017519323
##	91	92	93	94	95	96
##	0.640513373	0.394525274	0.240513373	-0.013498528	0.217519323	0.440513373
##	97	98	99	100	101	102
##	0.217519323	0.317519323	-0.536492577	0.117519323	-0.658409271	-0.220444974
##	103	104	105	106	107	108
##	0.133566927	0.502549076	-0.189427122	0.833566927	-0.374456875	1.202549076
##	109	110	111	112	113	114
##	0.702549076	-0.558409271	-0.443439023	-0.020444974	-0.266433073	-0.543439023
##	115	116	117	118	119	120
##	-1.335415221	-0.912421172	0.402549076	0.710572878	0.687578828	0.571531224
##	121	122	123	124	125	126
##	-0.512421172	-0.643439023	1.156560977	-0.197450924	-0.066433073	0.902549076
##	127	128	129	130	131	132
##	-0.297450924	-0.197450924	-0.166433073	1.148537175	0.779555026	0.856560977
##	133	134	135	136	137	138
##	-0.389427122	0.671531224	1.394525274	-0.112421172	-0.835415221	0.402549076
##	139	140	141	142	143	144
##	-0.297450924	-0.366433073	-0.835415221	-1.112421172	-0.220444974	-0.312421172
##	145	146	147	148	149	150
##	-0.958409271	-1.012421172	-0.320444974	-0.343439023	-0.812421172	0.002549076

Now we know the degrees of freedom for our model. We also have access to the residuals of our model. However, this is still not ideal. A lot of the information in the `model1` object is still several steps away from what we would need to

make statistical inferences. Fortunately, there are several ready-made functions to aid our interpretation.

The function you will most often encounter is `summary()`.

```
summary(model1)
```

```
##
## Call:
## lm(formula = Petal.Length ~ Petal.Width, data = iris)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-1.33542	-0.30347	-0.02955	0.25776	1.39453

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.08356	0.07297	14.85	<2e-16 ***
Petal.Width	2.22994	0.05140	43.39	<2e-16 ***

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4782 on 148 degrees of freedom
## Multiple R-squared:  0.9271, Adjusted R-squared:  0.9266
## F-statistic: 1882 on 1 and 148 DF,  p-value: < 2.2e-16
```

The summary function returns many useful pieces of information that we would need to draw statistical inferences:

- coefficient estimates, standard errors, t-statistics, and p-values;
- model degrees of freedom, R squared (explained variance), and model fit statistics;
- and more.

6.2.2 T-tests

For the most part, other statistical tests follow the same structure as our linear regression example: `y ~ x, data = df`

For example, an independent sample t-test to test if the difference in sepal length between “setosa” and “versicolor” plants is significant is ran as follows:

```
iris2 <- iris[iris$Species == "setosa" | iris$Species == "versicolor", ]

model2 <- t.test(Petal.Length ~ Species, data = iris2)

model2

##
##  Welch Two Sample t-test
##
## data:  Petal.Length by Species
## t = -39.493, df = 62.14, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group setosa and group versicolor is
## 95 percent confidence interval:
##  -2.939618 -2.656382
## sample estimates:
##      mean in group setosa mean in group versicolor
##                1.462                4.260
```

The output from the t-test function itself is informative enough in this case. Besides that, the approach is the same as for `lm()`.

6.2.3 ANOVA

An ANOVA to test if the differences between “setosa”, “versicolor”, and “virginica” in petal length is significant is performed as follows, using the `aov()` function:

```
model3 <- aov(Petal.Length ~ Species, data = iris)

summary(model3)

##              Df Sum Sq Mean Sq F value Pr(>F)
## Species      2  437.1   218.55    1180 <2e-16 ***
## Residuals    147   27.2     0.19
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

6.2.4 Correlations

To calculate the correlation between two variables, we use the `cor()` function, and do so without the model formula and data argument. Instead we simply enter our first variable and then our second variable we wish to check the correlation for:

```
model4 <- cor(iris$Sepal.Length, iris$Sepal.Width)
```

```
model4
```

```
## [1] -0.1175698
```

To find out if the correlation is significantly different from 0, we use the `cor.test()` function:

```
model5 <- cor.test(iris$Sepal.Length, iris$Sepal.Width)
```

```
model5
```

```
##
## Pearson's product-moment correlation
##
## data: iris$Sepal.Length and iris$Sepal.Width
## t = -1.4403, df = 148, p-value = 0.1519
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.27269325 0.04351158
## sample estimates:
## cor
## -0.1175698
```

6.2.5 Logistic Regression

A logistic regression is like a linear regression, only used to predict a binary variable (Yes/No, employed/unemployed, etc.) instead of a continuous variable (Sepal.Length, conscientiousness, etc.)

To perform a logistic regression, we make use of the `glm()` function. This function is a more general form of the `lm()` function, which can handle outcome variables that are not continuous with normally distributed errors.

However, to use this function properly, we will have to specify an additional argument, the `family` argument.

Without getting into detail about why and how we must change our family specification when we want to conduct a logistic regression. For now, simply know that you should make sure to specify `family = binomial()`, when you want to conduct logistic regression using `glm()`.

```

model6 <- glm(Species ~ Sepal.Length, data = iris2, family = binomial())

summary(model6)

##
## Call:
## glm(formula = Species ~ Sepal.Length, family = binomial(), data = iris2)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -27.831      5.434  -5.122 3.02e-07 ***
## Sepal.Length   5.140      1.007   5.107 3.28e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 138.629  on 99  degrees of freedom
## Residual deviance:  64.211  on 98  degrees of freedom
## AIC: 68.211
##
## Number of Fisher Scoring iterations: 6

```

6.2.6 Model Comparison

Suppose we have the following two models, and we want to know if adding the `Sepal.Width` variable and the interaction improve the fit of the model.

```

model11 <- lm(Petal.Length ~ Petal.Width, data = iris)

model17 <- lm(Petal.Length ~ Petal.Width*Sepal.Width, data = iris)

```

Note that we cannot merely look at the regression summary output, because we are looking at the change in fit as the result of multiple variables being added to the model.

```

summary(model17)

##
## Call:
## lm(formula = Petal.Length ~ Petal.Width * Sepal.Width, data = iris)
##
## Residuals:

```

```
##      Min      1Q   Median      3Q      Max
## -1.33452 -0.29091 -0.01051  0.21497  1.24780
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      2.281814   0.583452   3.911  0.00014 ***
## Petal.Width      2.132570   0.481634   4.428  1.85e-05 ***
## Sepal.Width     -0.362079   0.173239  -2.090  0.03835 *
## Petal.Width:Sepal.Width  0.007197   0.149522   0.048  0.96168
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.459 on 146 degrees of freedom
## Multiple R-squared:  0.9338, Adjusted R-squared:  0.9324
## F-statistic: 686.1 on 3 and 146 DF,  p-value: < 2.2e-16
```

To see which model fits better, we use the (confusingly named) `anova()` function.

```
anova(model1, model7)
```

```
## Analysis of Variance Table
##
## Model 1: Petal.Length ~ Petal.Width
## Model 2: Petal.Length ~ Petal.Width * Sepal.Width
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1     148  33.845
## 2     146  30.754   2    3.0903 7.3352 0.0009214 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

We can see that based on the p-value ($\text{Pr}(>F)$), the difference between the two models would be considered statistically significant at an alpha level of 0.05, since the p-value is below 0.05.

6.2.7 Advanced Statistics

We will not cover advanced statistical models in detail in these modules. We leave it for you to figure out how these more advanced models work. Fortunately, many will follow the same structure as the models we have discussed so far.

For example, suppose we wanted to run a mixed-effects (or multi-level, or hierarchical) regression model, we could use the `lmer()` function from the `lme4` package.

Random intercept model:

```
lmer(y ~ x + (1 | id), data = ...)
```

```
library(lme4)
```

```
## Loading required package: Matrix
```

```
## Warning: package 'Matrix' was built under R version 4.5.3
```

```
model8 <- lmer(Reaction ~ Days + (1 | Subject), data = sleepstudy)
```

```
summary(model8)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: Reaction ~ Days + (1 | Subject)
## Data: sleepstudy
##
## REML criterion at convergence: 1786.5
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -3.2257 -0.5529  0.0109  0.5188  4.2506
##
## Random effects:
## Groups   Name            Variance Std.Dev.
## Subject (Intercept) 1378.2    37.12
## Residual                960.5    30.99
## Number of obs: 180, groups: Subject, 18
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept) 251.4051    9.7467   25.79
## Days        10.4673    0.8042   13.02
##
## Correlation of Fixed Effects:
##      (Intr)
## Days -0.371
```

START EXERCISE 4